

Prueba de Caja Blanca

“Sistema Digital de Gestión de Criadero: Hoptolt”

Integrantes:

Panchez Jacome Darwin Javier
Robalino Curay Cristian Isaak
Proaño Váscquez José Ignacio

Fecha: 2025-07-07

Prueba caja blanca de REQ001 – Registrar datos de jaula

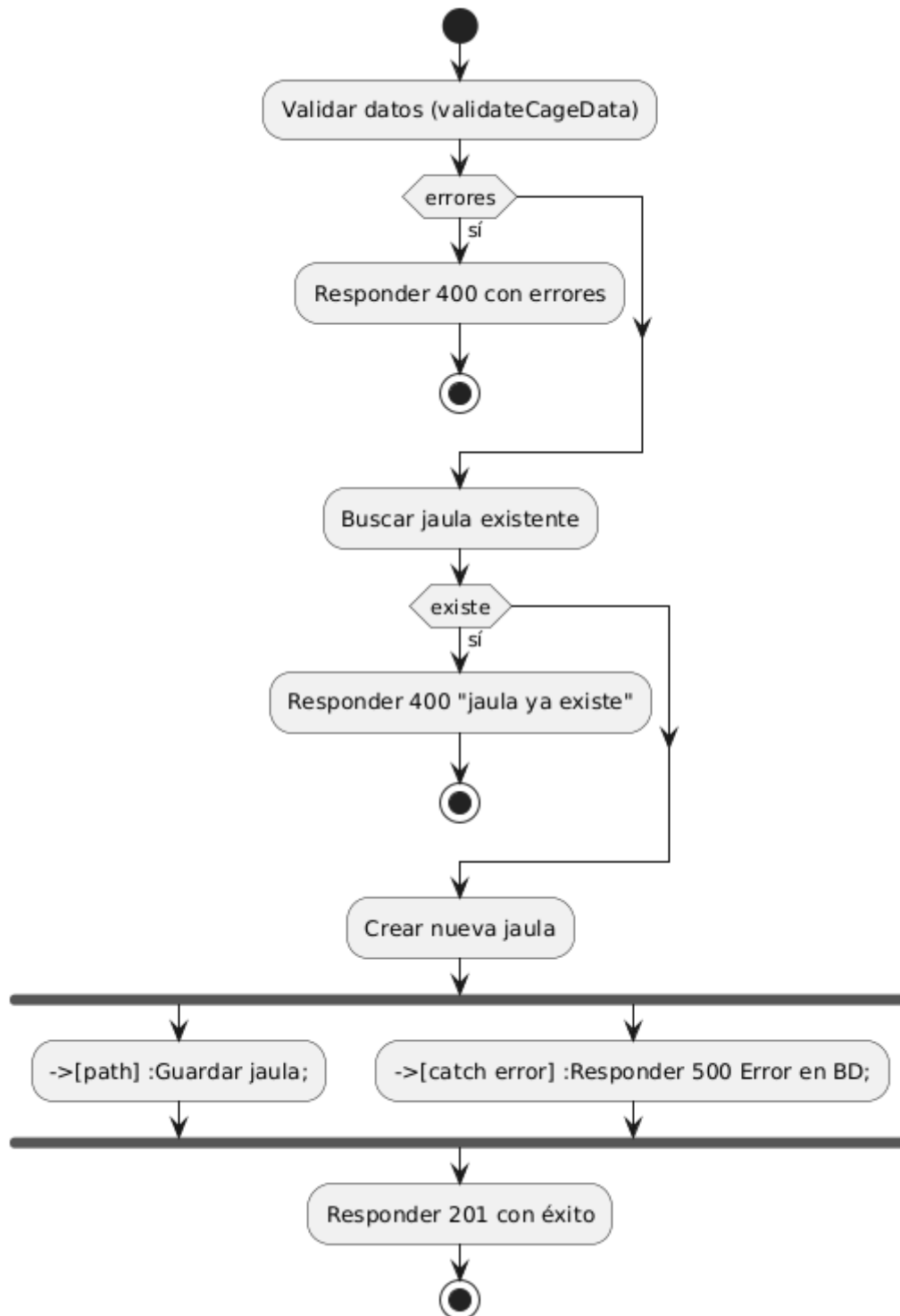
1. CÓDIGO FUENTE

```
exports.registerCage = async (req, res) => {
  const errors = validateCageData(req.body);
  if (errors.length > 0) return res.status(400).json({ errors });

  try {
    const exists = await Cage.findOne({ number: req.body.number
});
    if (exists) return res.status(400).json({ errors: ['El número
de jaula ya existe.'] });

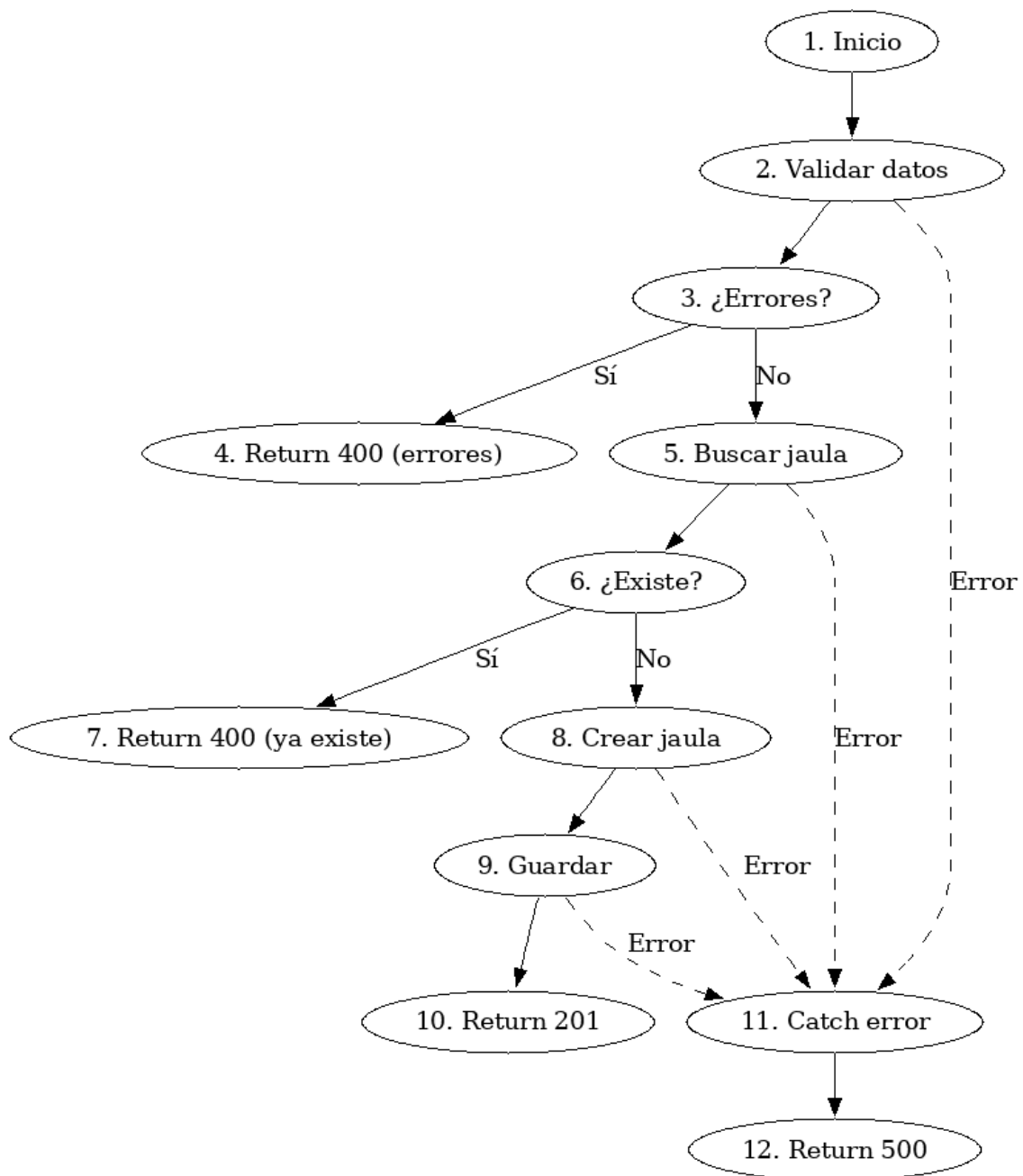
    const cage = new Cage(req.body);
    await cage.save();
    res.status(201).json({ message: 'Jaula registrada con éxito',
cage });
  } catch (err) {
    res.status(500).json({ message: 'Error en la base de datos',
error: err.message });
  }
};
```

2. DIAGRAMA DE FLUJO (DF)



3. GRAFO DE FLUJO (GF)

Nodos numerados para el grafo:



Aristas (A):

- (1→2), (2→3), (3→4), (3→5), (5→6), (6→7), (6→8), (8→9), (9→10), (2→11), (5→11), (8→11), (9→11), (11→12)

4. IDENTIFICACIÓN DE LAS RUTAS (Camino basico)

RUTAS

Rutas posibles desde el grafo:

- **R1:** 1 → 2 → 3 (con error) → 4
- **R2:** 1 → 2 → 3 (sin error) → 5 → 6 (jaula ya existe) → 7
- **R3:** 1 → 2 → 3 (sin error) → 5 → 6 (no existe) → 8 → 9 → 10
- **R4:** 1 → 2 → 3 (sin error) → 5 → 6 (no existe) → 8 → Error → 11 → 12
- **R5:** 1 → 2 → Error → 11 → 12
- **R6:** 1 → 2 → 3 (sin error) → Error en búsqueda → 11 → 12

5. COMPLEJIDAD CICLOMÁTICA

Se puede calcular de las siguientes formas:

- $V(G) = \text{número de nodos predicados(decisiones)} + 1$
 $V(G) = 3 + 1 = 4$
- $V(G) = A - N + 2$
 $V(G) = 14 - 12 + 2 = 2 + 2 = 4$

DONDE:

P: Número de nodos predicado

A: Número de aristas

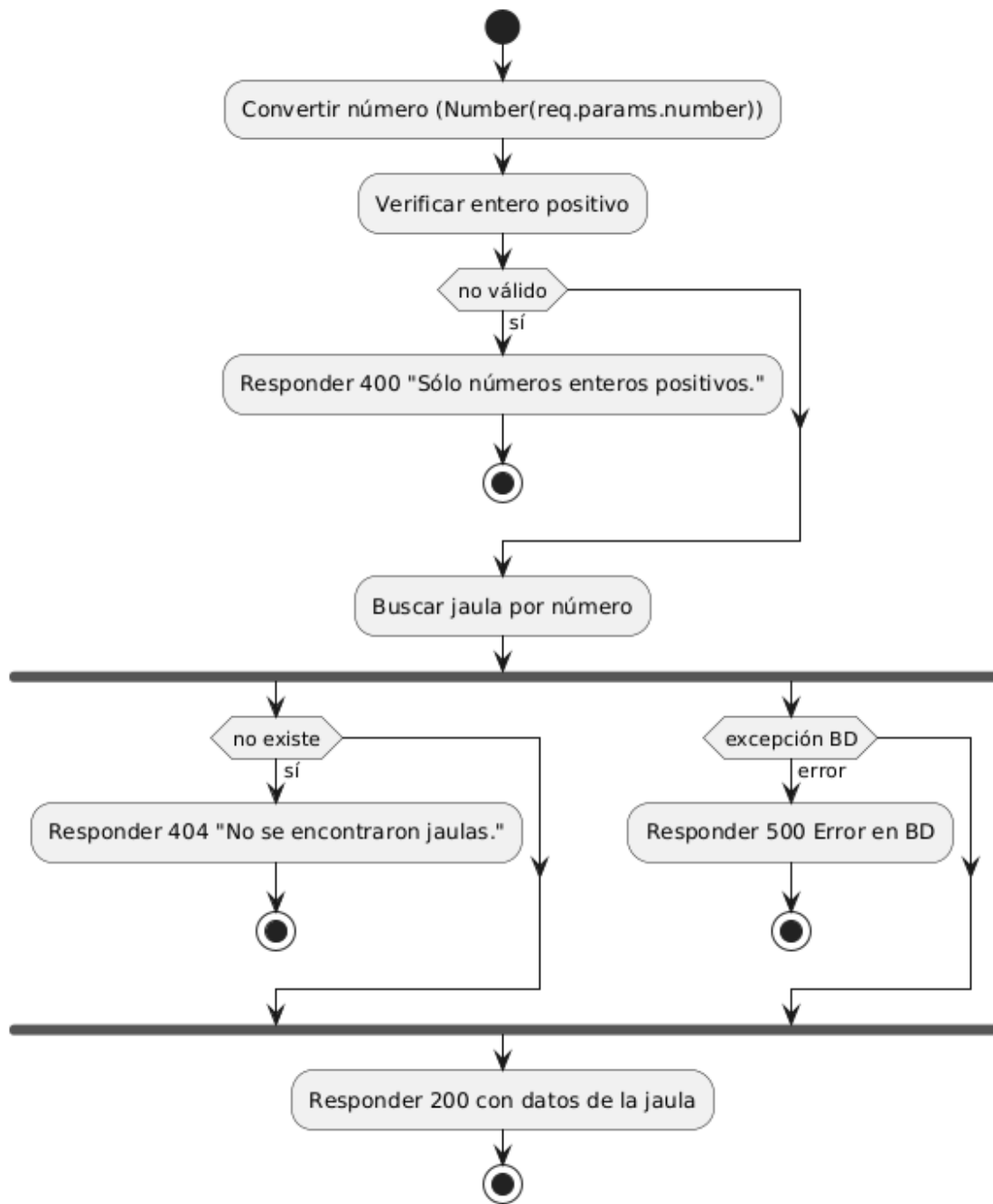
N: Número de nodos

Prueba caja blanca de REQ002 – Consultar datos de jaula

6. CÓDIGO FUENTE

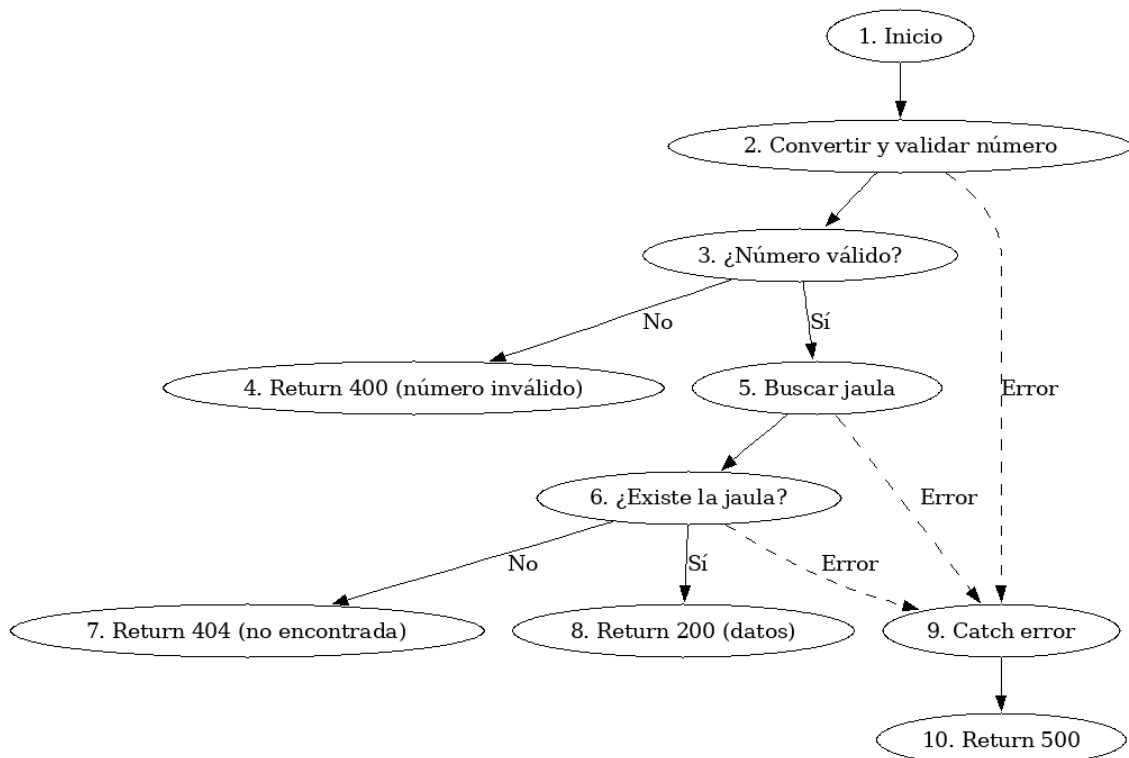
```
exports.getCage = async (req, res) => {
  const number = Number(req.params.number);
  if (!Number.isInteger(number) || number <= 0) {
    return res.status(400).json({ message: 'Sólo números enteros positivos.' });
  }
  try {
    const cage = await Cage.findOne({ number });
    if (!cage) return res.status(404).json({ message: 'No se encontraron jaulas.' });
    res.json(cage);
  } catch (err) {
    res.status(500).json({ message: 'Error en la base de datos', error: err.message });
  }
};
```

7. DIAGRAMA DE FLUJO (DF)



8. GRAFO DE FLUJO (GF)

Nodos numerados para el grafo:



Aristas (A):

- (1→2), (2→3), (3→4), (3→5), (5→6), (6→7), (6→8),
- (2→9), (5→9), (6→9), (9→10)

9. IDENTIFICACIÓN DE LAS RUTAS (Camino basico)

RUTAS

Rutas posibles desde el grafo:

- **R1:** 1 → 2 → 3 (invalid) → 4
- **R2:** 1 → 2 → 3 (valid) → 5 → 6 (no existe) → 7
- **R3:** 1 → 2 → 3 (valid) → 5 → 6 (existe) → 8
- **R4:** 1 → 2 → error → 9 → 10
- **R5:** 1 → 2 → 3 (valid) → 5 → error → 9 → 10
- **R6:** 1 → 2 → 3 (valid) → 5 → 6 → error → 9 → 10

10. COMPLEJIDAD CICLOMÁTICA

Se puede calcular de las siguientes formas:

- $V(G) = \text{número de nodos predicados(decisiones)} + 1$
 Número de nodos predicados(decisiones) = (validación número, existencia jaula) = 2
 $V(G) = 2 + 1 = 3$
- $V(G) = A - N + 2$
 $V(G) = 11 - 10 + 2 = 1 + 2 = 3$

DONDE:

P: Número de nodos predicado

A: Número de aristas

N: Número de nodos

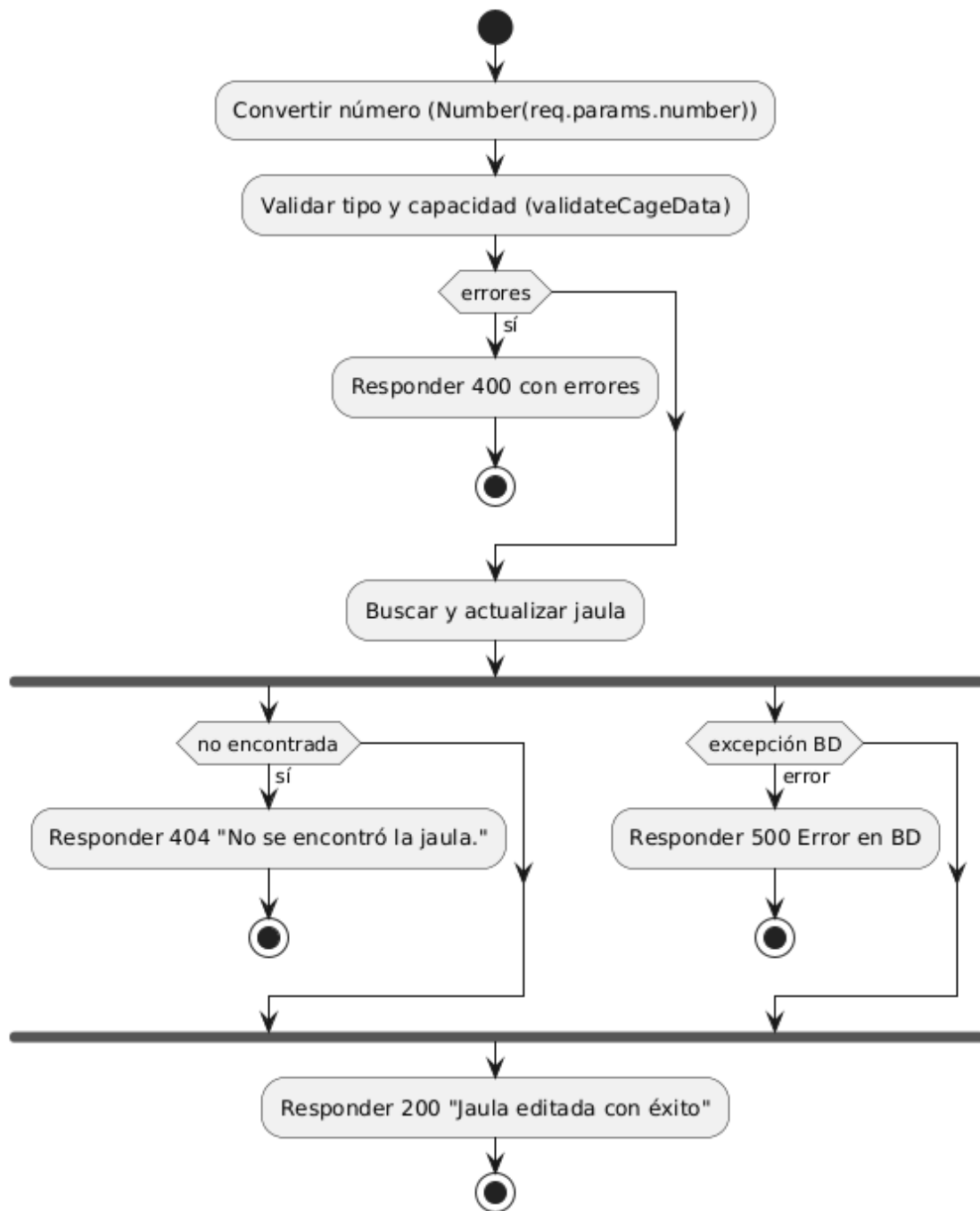
Prueba caja blanca de REQ003 – Editar datos de jaula

11. CÓDIGO FUENTE

```
exports.editCage = async (req, res) => {
  const number = Number(req.params.number);
  const errors = validateCageData({ ...req.body, number }, false);
  if (errors.length > 0) return res.status(400).json({ errors });

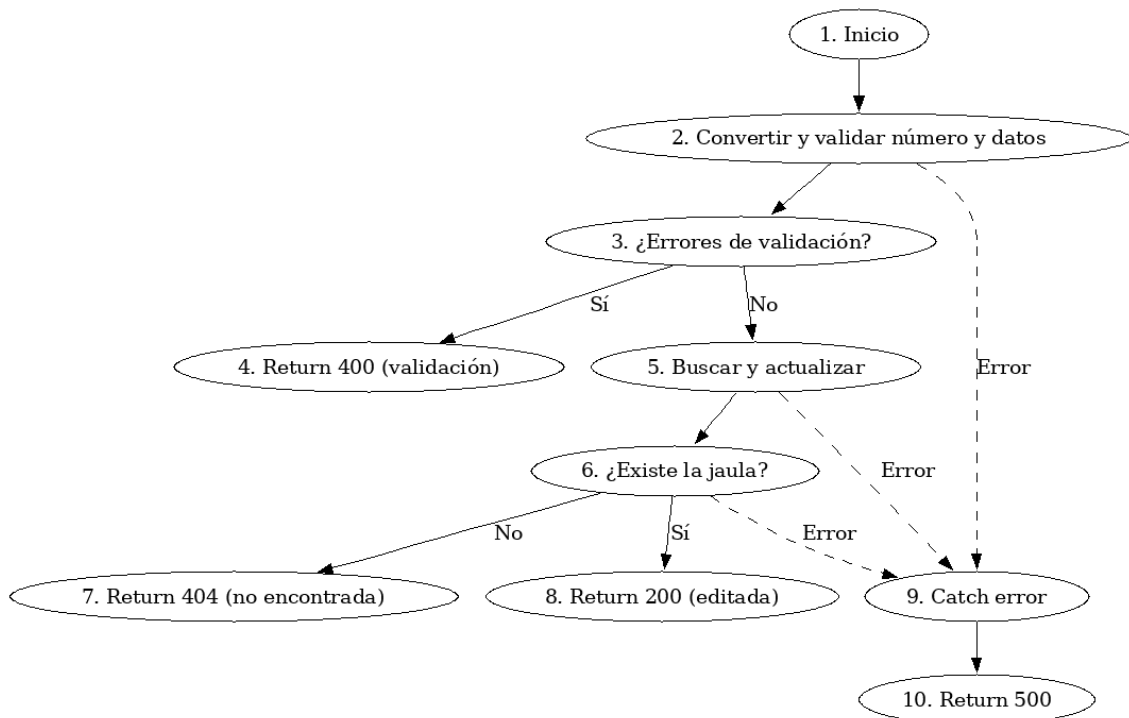
  try {
    const cage = await Cage.findOneAndUpdate(
      { number },
      { type: req.body.type, capacity: req.body.capacity },
      { new: true }
    );
    if (!cage) return res.status(404).json({ message: 'No se encontró la jaula.' });
    res.json({ message: 'Jaula editada con éxito', cage });
  } catch (err) {
    res.status(500).json({ message: 'Error en la base de datos', error: err.message });
  }
};
```

12. DIAGRAMA DE FLUJO (DF)



13. GRAFO DE FLUJO (GF)

Nodos numerados para el grafo:



Aristas (A):

- (1→2), (2→3), (3→4), (3→5), (5→6), (6→7), (6→8), (2→9), (5→9), (6→9), (9→10)

14. IDENTIFICACIÓN DE LAS RUTAS (Camino basico)

RUTAS

Rutas posibles desde el grafo:

- **R1:** 1 → 2 → 3 (errores) → 4
- **R2:** 1 → 2 → 3 (sin errores) → 5 → 6 (no existe) → 7
- **R3:** 1 → 2 → 3 (sin errores) → 5 → 6 (existe) → 8
- **R4:** 1 → 2 → error → 9 → 10
- **R5:** 1 → 2 → 3 → 5 → error → 9 → 10
- **R6:** 1 → 2 → 3 → 5 → 6 → error → 9 → 10

15. COMPLEJIDAD CICLOMÁTICA

Se puede calcular de las siguientes formas:

- $V(G) = \text{número de nodos predicados(decisiones)} + 1$
 Número de nodos predicados(decisiones) = (validación, existencia) = 2
 $V(G) = 2 + 1 = 3$
- $V(G) = A - N + 2$
 $V(G) = 11 - 10 + 2 = 1 + 2 = 3$

DONDE:

P: Número de nodos predicado

A: Número de aristas

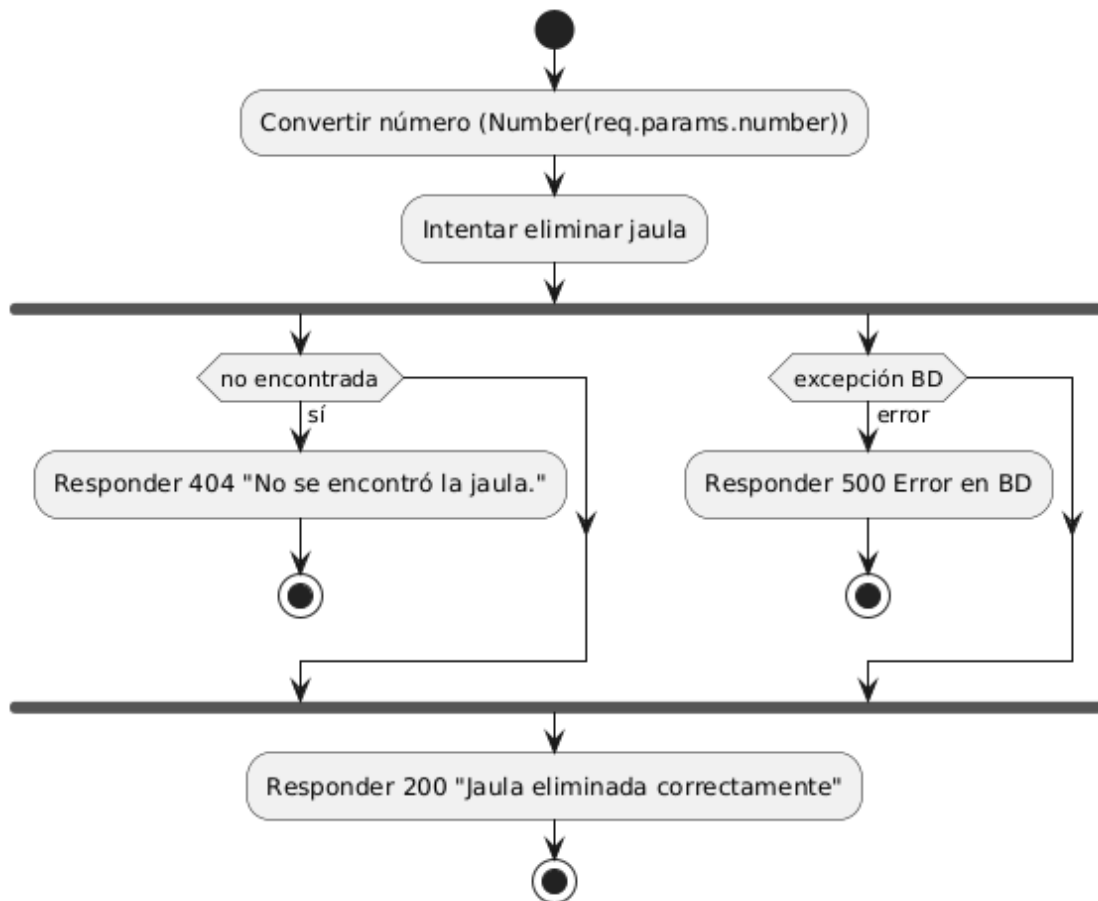
N: Número de nodos

Prueba caja blanca de REQ004 – Eliminar jaula

16. CÓDIGO FUENTE

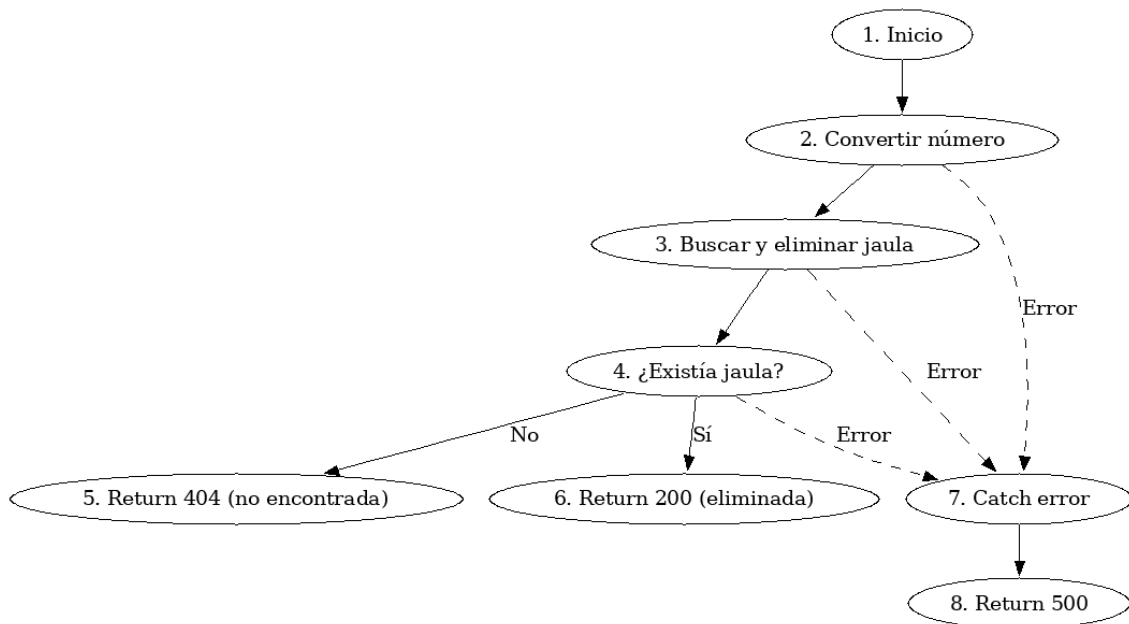
```
exports.deleteCage = async (req, res) => {  
  const number = Number(req.params.number);  
  try {  
    const cage = await Cage.findOneAndDelete({ number });  
    if (!cage) return res.status(404).json({ message: 'No se  
encontró la jaula.' });  
    res.json({ message: 'Jaula eliminada correctamente.' });  
  } catch (err) {  
    res.status(500).json({ message: 'Error en la base de datos',  
error: err.message });  
  }  
};
```

17. DIAGRAMA DE FLUJO (DF)



18. GRAFO DE FLUJO (GF)

Nodos numerados para el grafo:



Aristas (A):

- (1→2), (2→3), (3→4), (4→5), (4→6),
- (2→7), (3→7), (4→7), (7→8)

19. IDENTIFICACIÓN DE LAS RUTAS (Camino basico)

RUTAS

Rutas posibles desde el grafo:

- **R1:** 1 → 2 → 3 → 4 (no existe) → 5
- **R2:** 1 → 2 → 3 → 4 (existe) → 6
- **R3:** 1 → 2 → error → 7 → 8
- **R4:** 1 → 2 → 3 → error → 7 → 8
- **R5:** 1 → 2 → 3 → 4 → error → 7 → 8

20. COMPLEJIDAD CICLOMÁTICA

Se puede calcular de las siguientes formas:

- $V(G) = \text{número de nodos predicados(decisiones)} + 1$
 Número de nodos predicados(decisiones) = (existencia jaula, excepción BD) = 2
 $V(G) = 2 + 1 = 3$
- $V(G) = A - N + 2$
 $V(G) = 9 - 8 + 2 = 1 + 2 = 3$

DONDE:

P: Número de nodos predicado

A: Número de aristas

N: Número de nodos

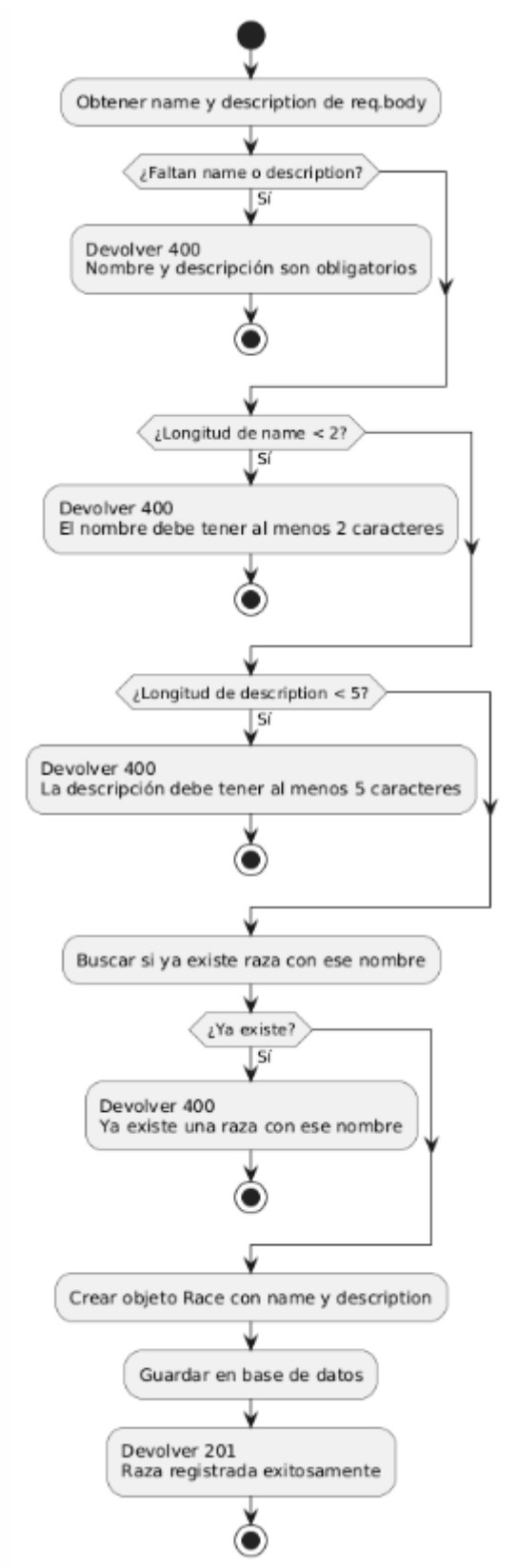
Prueba caja blanca de REQ005 – Registrar raza

21. CÓDIGO FUENTE

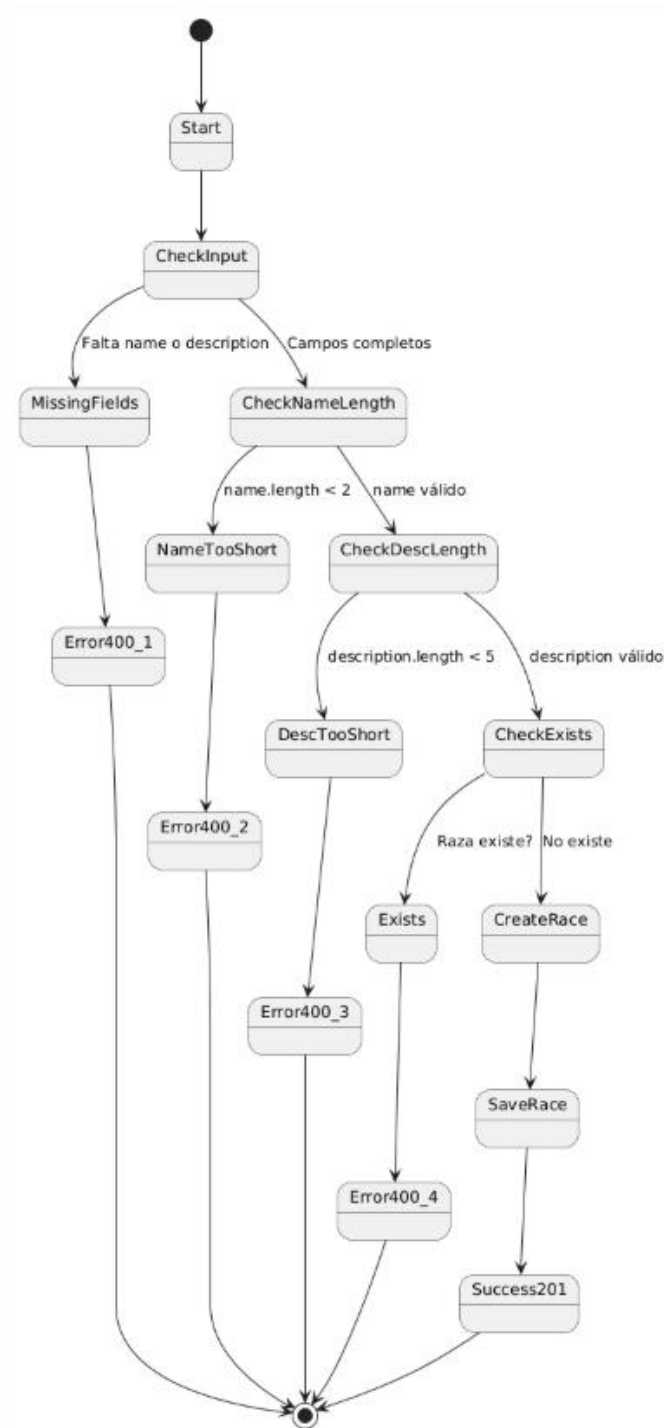
```
exports.registerRace = async (req, res) => {
  const { name, description } = req.body;
  if (!name || !description) {
    return res.status(400).json({ message: 'Nombre y descripción son obligatorios.' });
  }
  if (name.length < 2) {
    return res.status(400).json({ message: 'El nombre debe tener al menos 2 caracteres.' });
  }
  if (description.length < 5) {
    return res.status(400).json({ message: 'La descripción debe tener al menos 5 caracteres.' });
  }

  try {
    const exists = await Race.findOne({ name: name.trim() });
    if (exists) {
      return res.status(400).json({ message: 'Ya existe una raza con ese nombre.' });
    }
    const race = new Race({ name: name.trim(), description: description.trim() });
    await race.save();
    res.status(201).json({ message: 'Raza registrada exitosamente.', race });
  } catch (error) {
    res.status(500).json({ message: 'Error al registrar la raza.', error: error.message });
  }
};
```

22. DIAGRAMA DE FLUJO (DF)



23. GRAFO DE FLUJO (GF)



24. IDENTIFICACIÓN DE LAS RUTAS (Camino basico)

- **R1:** 1 → 2 → 3 → 4
Caso donde falta name o description → error 400 por campos obligatorios.
- **R2:** 1 → 2 → 5 → 6 → 7
Campos completos → name demasiado corto → error 400 nombre corto.
- **R3:** 1 → 2 → 5 → 8 → 9 → 10
Campos completos y name válido → descripción demasiado corta → error 400 descripción corta.
- **R4:** 1 → 2 → 5 → 8 → 11 → 12 → 13
Campos completos, name y descripción válidos → raza ya existe → error 400 raza duplicada.
- **R5:** 1 → 2 → 5 → 8 → 11 → 14 → 15 → 16
Flujo exitoso: campos completos y válidos, no existe raza, se crea y guarda, retorna éxito.

25. COMPLEJIDAD CICLOMÁTICA

Se puede calcular de las siguientes formas:

- $V(G) = \text{número de nodos predicados (decisiones)} + 1$
 $V(G) = 4 + 1 = 5$
- $V(G) = A - N + 2$
 $V(G) = 19 - 16 + 2 = 3 + 2 = 5$

DONDE:

P: Número de nodos predicado = 4

A: Número de aristas = 19

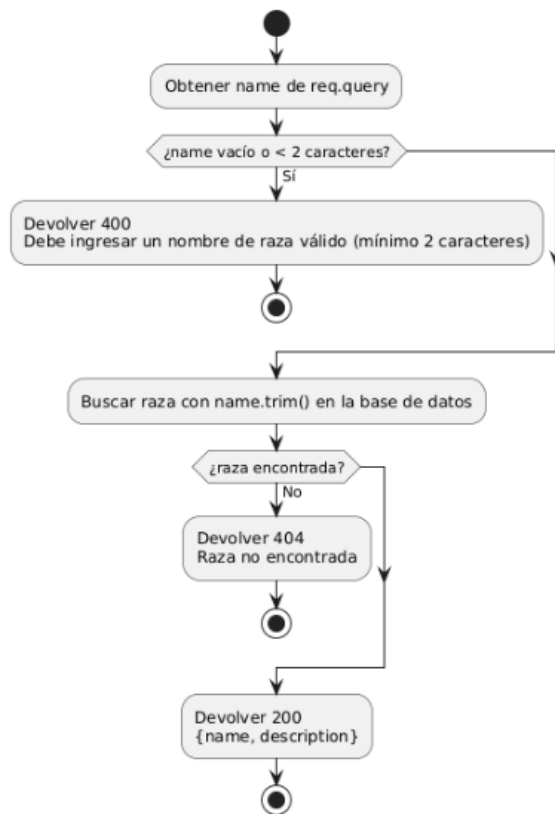
N: Número de nodos = 16

Prueba caja blanca de REQ006 – Consultar raza

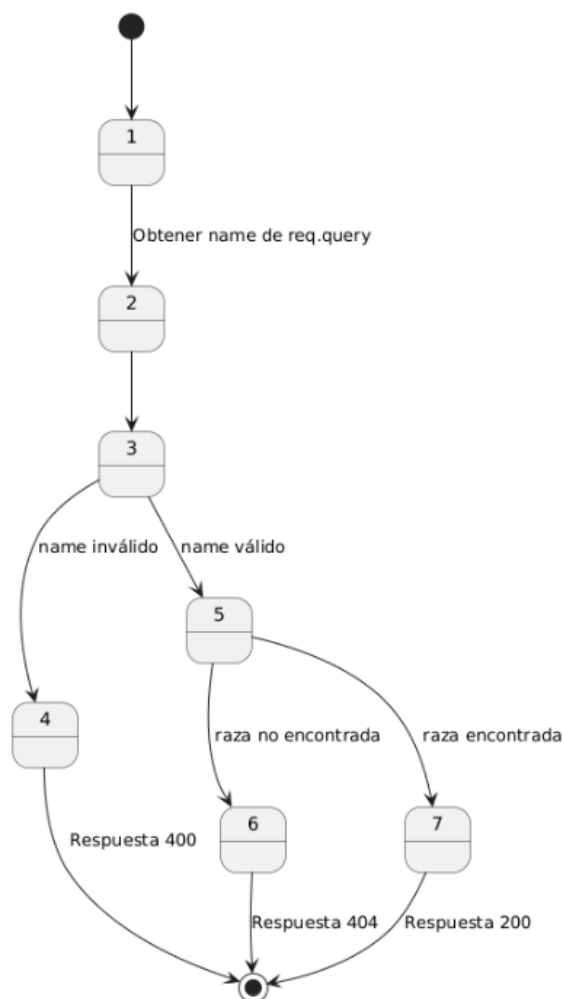
1. CÓDIGO FUENTE

```
exports.getRaceByName = async (req, res) => {
  const { name } = req.query;
  if (!name || name.trim().length < 2) {
    return res.status(400).json({ message: 'Debe ingresar un nombre de raza válido (mínimo 2 caracteres).' });
  }
  try {
    const race = await Race.findOne({ name: name.trim() });
    if (!race) {
      return res.status(404).json({ message: 'Raza no encontrada.' });
    }
    res.status(200).json({ name: race.name, description: race.description });
  } catch (error) {
    res.status(500).json({ message: 'Error al consultar la raza.', error: error.message });
  }
};
```

2. DIAGRAMA DE FLUJO (DF)



3. GRAFO DE FLUJO (GF)



4. IDENTIFICACIÓN DE LAS RUTAS (Camino basico)

- **R1:** $1 \rightarrow 2 \rightarrow 3 \rightarrow 4$
Nombre inválido (vacío o menos de 2 caracteres) $\rightarrow$ error 400.
- **R2:** $1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 6$
Nombre válido $\rightarrow$ búsqueda en BD $\rightarrow$ raza no encontrada $\rightarrow$ error 404.
- **R3:** $1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 7$
Nombre válido $\rightarrow$ búsqueda en BD $\rightarrow$ raza encontrada $\rightarrow$ respuesta 200 con datos.

5. COMPLEJIDAD CICLOMÁTICA

Se puede calcular de las siguientes formas:

- $V(G) = \text{número de nodos prediados (decisiones)} + 1$
 $V(G) = 2 + 1 = 3$
- $V(G) = A - N + 2$
 $V(G) = 7 - 9 + 2 = 0 + 2 = 2$

DONDE:

P: Número de nodos prediados = 2

A: Número de aristas = 7

N: Número de nodos = 9

Prueba caja blanca de REQ007 – Editar raza

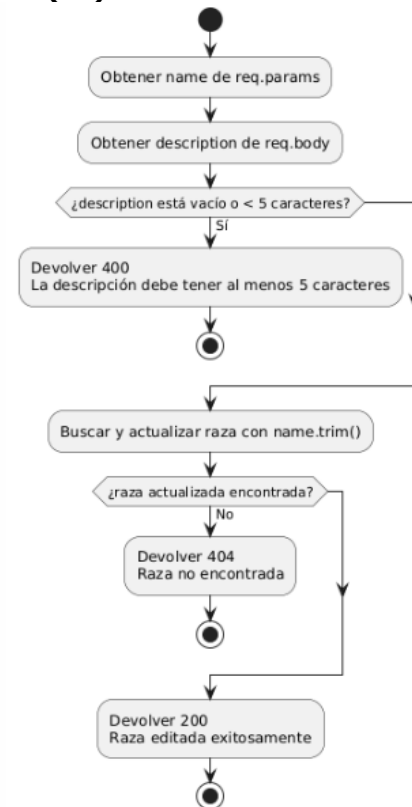
1. CÓDIGO FUENTE

```
exports.editRaceDescription = async (req, res) => {
  const { name } = req.params;
  const { description } = req.body;

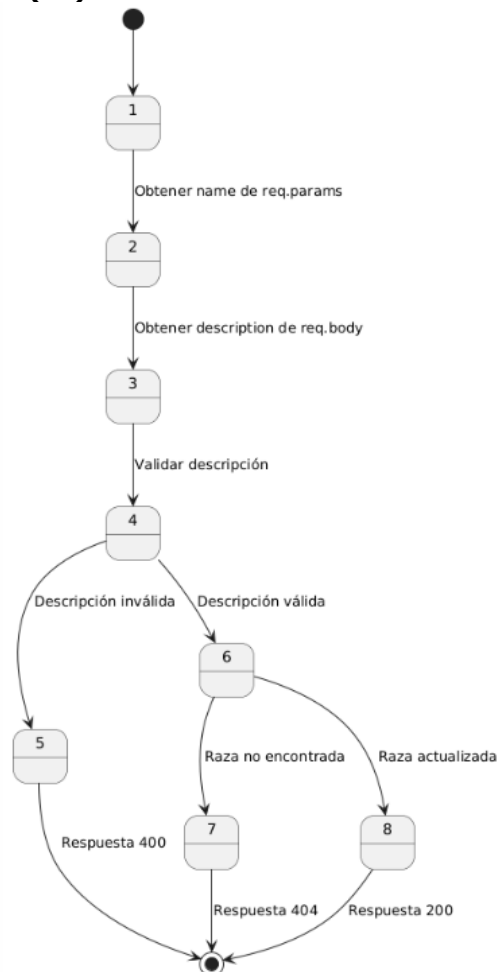
  if (!description || description.trim().length < 5) {
    return res.status(400).json({ message: 'La descripción debe tener al menos 5 caracteres.' });
  }

  try {
    const updatedRace = await Race.findOneAndUpdate(
      { name: name.trim() },
      { description: description.trim() },
      { new: true }
    );
    if (!updatedRace) {
      return res.status(404).json({ message: 'Raza no encontrada.' });
    }
    res.status(200).json({ message: 'Raza editada exitosamente.', race: updatedRace });
  } catch (error) {
    res.status(500).json({ message: 'Error al editar la raza.', error: error.message });
  }
};
```

2. DIAGRAMA DE FLUJO (DF)



3. GRAFO DE FLUJO (GF)



4. IDENTIFICACIÓN DE LAS RUTAS (Camino basico)

- **R1:** $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$

Descripción inválida (vacía o menos de 5 caracteres) $\rightarrow$ error 400.

- **R2:** $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 6 \rightarrow 7$

Descripción válida $\rightarrow$ buscar y actualizar raza $\rightarrow$ raza no encontrada $\rightarrow$ error 404.

- **R3:** $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 6 \rightarrow 8$

Descripción válida $\rightarrow$ buscar y actualizar raza $\rightarrow$ éxito $\rightarrow$ respuesta 200.

5. COMPLEJIDAD CICLOMÁTICA

Se puede calcular de las siguientes formas:

- $V(G) = \text{número de nodos predcados (decisiones)} + 1$
 $V(G) = 2 + 1 = 3$
- $V(G) = A - N + 2$
 $V(G) = 8 - 9 + 2 = 1$

DONDE:

P: Número de nodos predcado = 2

A: Número de aristas = 8

N: Número de nodos = 9

Prueba caja blanca de REQ008 – Eliminar raza

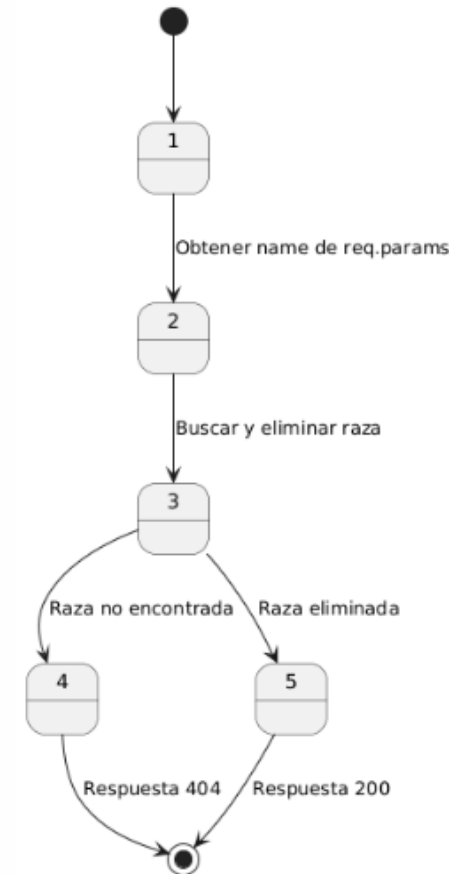
1. CÓDIGO FUENTE

```
exports.deleteRace = async (req, res) => {
  const { name } = req.params;
  try {
    const deleted = await Race.findOneAndDelete({ name: name.trim()
  });
    if (!deleted) {
      return res.status(404).json({ message: 'Raza no encontrada.'
    });
  }
    res.status(200).json({ message: 'Raza eliminada con éxito.' });
  } catch (error) {
    res.status(500).json({ message: 'Error al eliminar la raza.',
error: error.message });
  }
};
```

2. DIAGRAMA DE FLUJO (DF)



3. GRAFO DE FLUJO (GF)



4. IDENTIFICACIÓN DE LAS RUTAS (Camino basico)

- **R1:** $1 \rightarrow 2 \rightarrow 3 \rightarrow 4$

No se encuentra la raza $\rightarrow$ error 404.

- **R2:** $1 \rightarrow 2 \rightarrow 3 \rightarrow 5$

Raza encontrada y eliminada con éxito $\rightarrow$ respuesta 200.

5. COMPLEJIDAD CICLOMÁTICA

Se puede calcular de las siguientes formas:

- $V(G) = \text{número de nodos prediados (decisiones)} + 1$
 $V(G) = 1 + 1 = 2$
- $V(G) = A - N + 2$
 $V(G) = 5 - 6 + 2 = 1$

DONDE:

P: Número de nodos prediados = 1

A: Número de aristas = 5

N: Número de nodos = 6

Prueba caja blanca de REQ009 – Registrar datos de conejos

26. CÓDIGO FUENTE

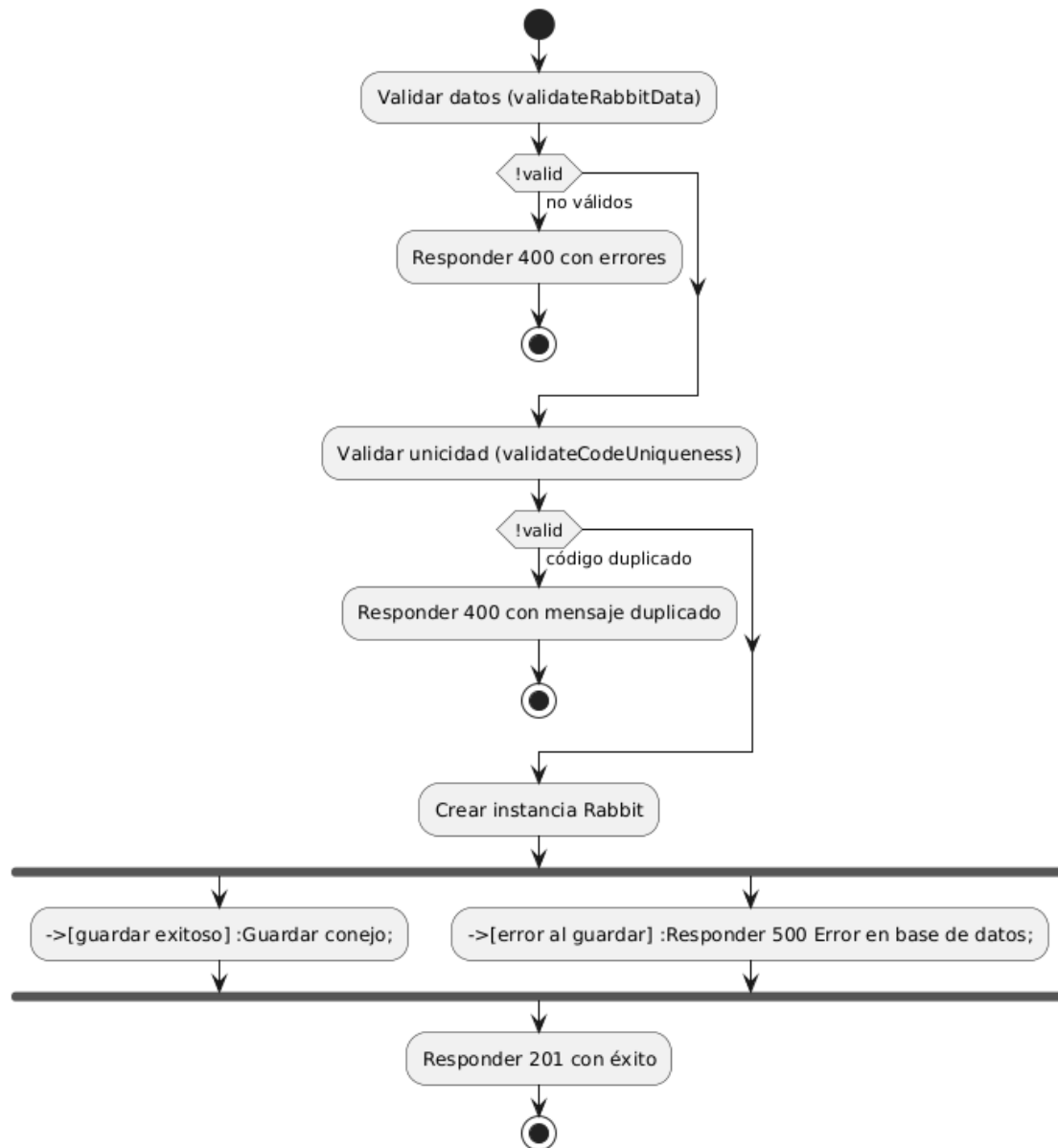
```
exports.registerRabbit = async (req, res) => {
  console.log('Datos recibidos:', req.body);
  const { race, code, sex, age, weight, purpose } = req.body;

  try {
    // Validar datos
    const validation = await validateRabbitData(req.body);
    if (!validation.valid) {
      return res.status(400).json({ errors: validation.errors });
    }

    // Validar unicidad del código
    const codeValidation = await validateCodeUniqueness(code);
    if (!codeValidation.valid) {
      return res.status(400).json({ message:
codeValidation.message });
    }

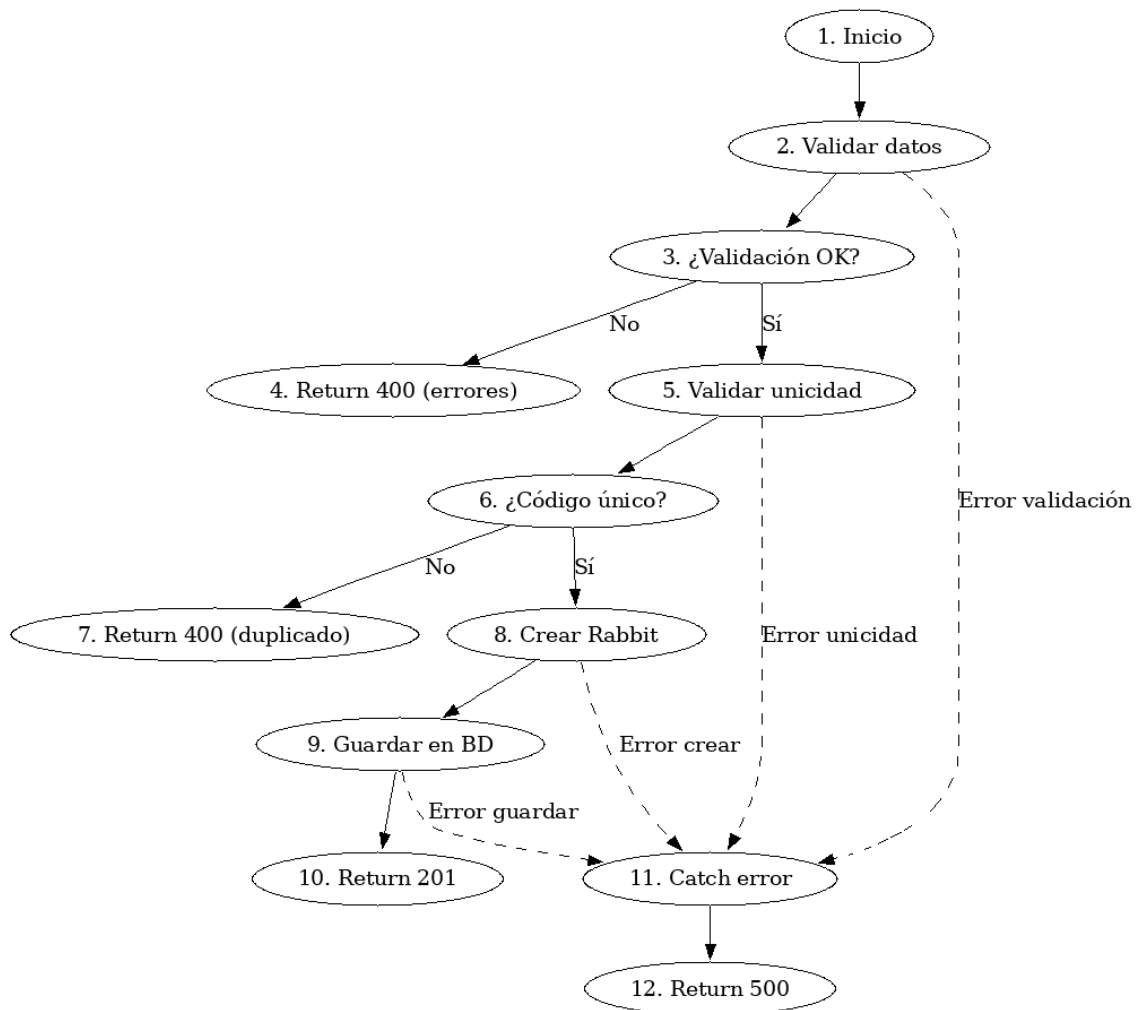
    const newRabbit = new Rabbit({ race, code, sex, age, weight,
purpose });
    await newRabbit.save();
    res.status(201).json({ message: 'Conejo registrado
exitosamente', rabbit: newRabbit });
  } catch (error) {
    console.error(error);
    res.status(500).json({ message: 'Error al registrar el
conejo', error: error.message });
  }
};
```

27. DIAGRAMA DE FLUJO (DF)



28. GRAFO DE FLUJO (GF)

Nodos numerados para el grafo:



Aristas (A):

- (1→2), (2→3), (3→4), (3→5), (5→6), (6→7), (6→8), (8→9), (9→10), (2→11), (5→11), (8→11), (9→11), (11→12)

29. IDENTIFICACIÓN DE LAS RUTAS (Camino basico)

RUTAS

Rutas posibles desde el grafo:

- **R1:** 1 → 2 → 3 (no válido) → 4
- **R2:** 1 → 2 → 3 (válido) → 5 → 6 (no único) → 7
- **R3:** 1 → 2 → 3 (válido) → 5 → 6 (único) → 8 → 9 → 10
- **R4:** 1 → 2 → 3 (válido) → 5 → 6 (único) → 8 → Error → 11 → 12
- **R5:** 1 → 2 → Error validación → 11 → 12
- **R6:** 1 → 2 → 3 (válido) → 5 → Error unicidad → 11 → 12

30. COMPLEJIDAD CICLOMÁTICA

Se puede calcular de las siguientes formas:

- $V(G) = \text{número de nodos predicados(decisiones)} + 1$
 Número de nodos predicados(decisiones) = (validación datos, existencia conejo, error de registro) = 3
 $V(G) = 3 + 1 = 4$
- $V(G) = A - N + 2$
 $V(G) = 14 - 12 + 2 = 2 + 2 = 4$

DONDE:

P: Número de nodos predicado

A: Número de aristas

N: Número de nodos

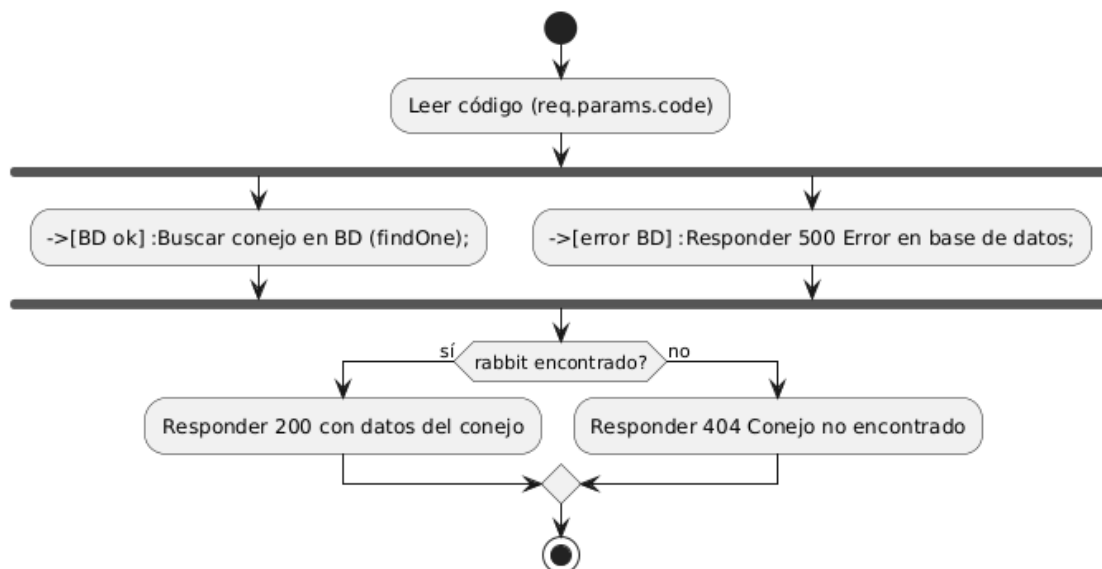
Prueba caja blanca de REQ010 – Consultar datos de conejo

31. CÓDIGO FUENTE

```
exports.getRabbit = async (req, res) => {
  const { code } = req.params;

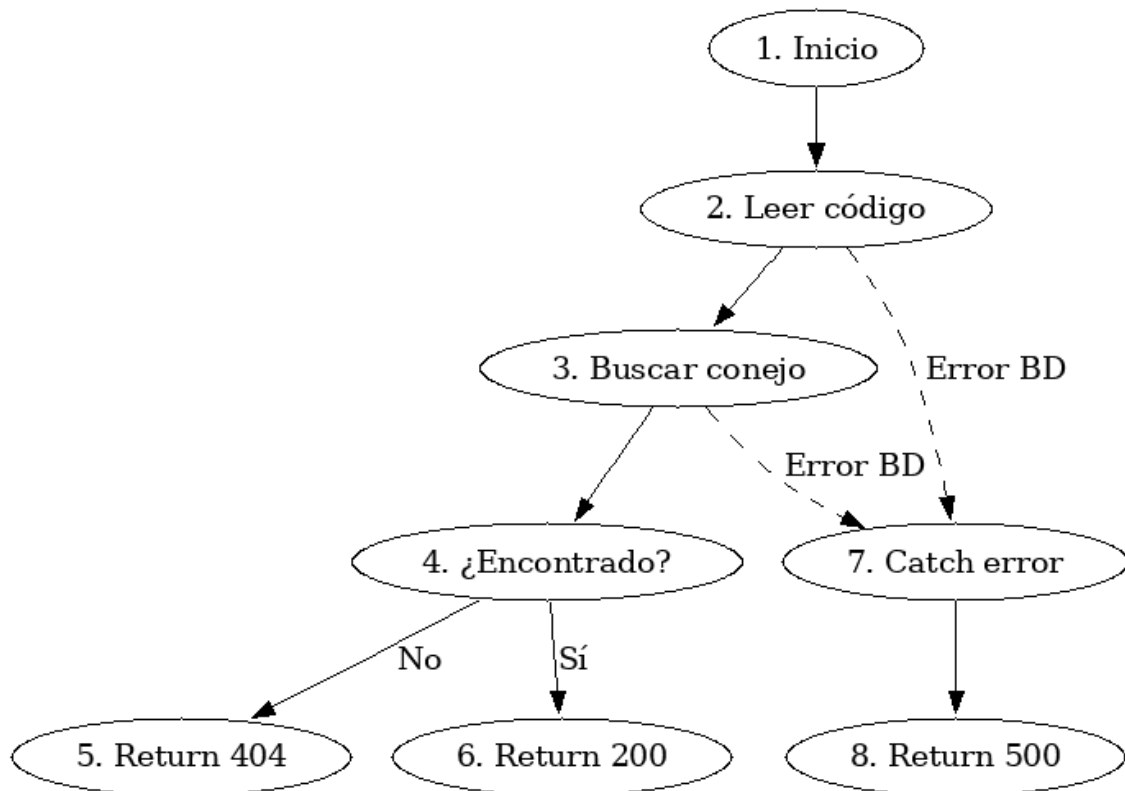
  try {
    const rabbit = await Rabbit.findOne({ code });
    if (!rabbit) {
      return res.status(404).json({ message: 'Conejo no encontrado' });
    }
    res.status(200).json(rabbit);
  } catch (error) {
    res.status(500).json({ message: 'Error al obtener el conejo', error: error.message });
  }
};
```

32. DIAGRAMA DE FLUJO (DF)



33. GRAFO DE FLUJO (GF)

Nodos numerados para el grafo:



Aristas (A):

- (1→2), (2→3), (3→4), (4→5), (4→6), (2→7), (3→7), (7→8)

34. IDENTIFICACIÓN DE LAS RUTAS (Camino basico)

RUTAS

Rutas posibles desde el grafo:

- **R1:** 1 → 2 (error BD) → 7 → 8
- **R2:** 1 → 2 → 3 (error BD) → 7 → 8
- **R3:** 1 → 2 → 3 → 4 (no encontrado) → 5
- **R4:** 1 → 2 → 3 → 4 (encontrado) → 6

35. COMPLEJIDAD CICLOMÁTICA

Se puede calcular de las siguientes formas:

- $V(G) = \text{número de nodos predicados(decisiones)} + 1$
 Número de nodos predicados(decisiones) = (Encontrado) = 1
 $V(G) = 1 + 1 = 2$
- $V(G) = A - N + 2$
 $V(G) = 8 - 8 + 2 = 0 + 2 = 2$

DONDE:

P: Número de nodos predicado

A: Número de aristas

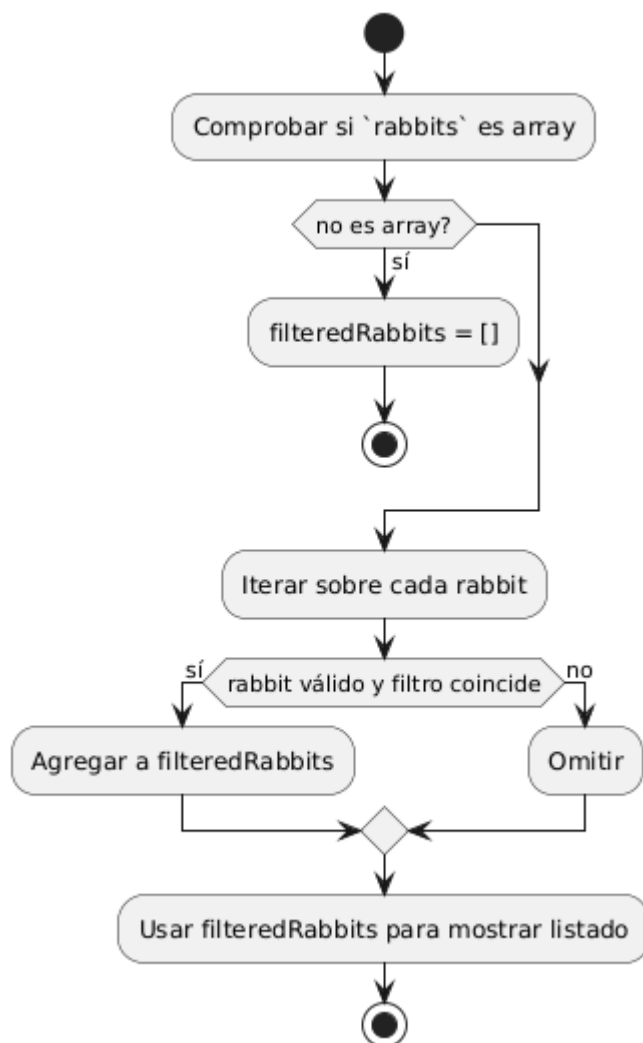
N: Número de nodos

Prueba caja blanca de REQ011 – Filtrar conejos por raza

36. CÓDIGO FUENTE

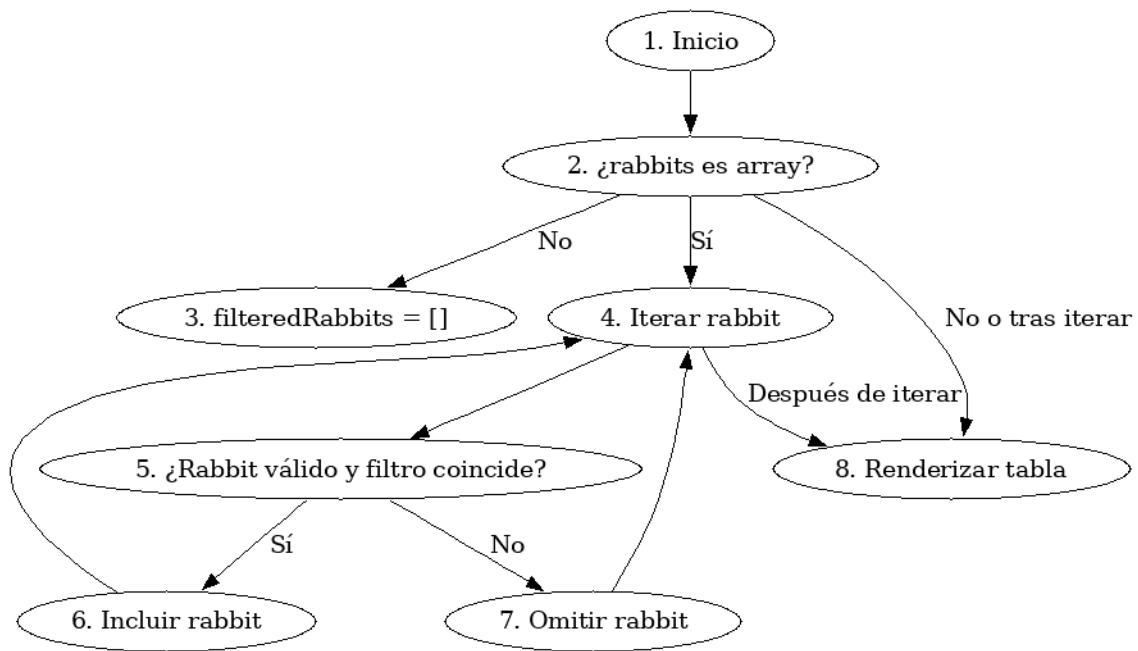
```
const filteredRabbits = Array.isArray(rabbits)
  ? rabbits.filter(rabbit =>
    rabbit &&
    rabbit.code &&
    (rabbit.code.toLowerCase().includes(filter.toLowerCase().trim()) ||
    rabbit.race.toLowerCase().includes(filter.toLowerCase().trim())))
  : [];
```

37. DIAGRAMA DE FLUJO (DF)



38. GRAFO DE FLUJO (GF)

Nodos numerados para el grafo:



Aristas (A):

- (1→2), (2→3), (2→4), (4→5), (5→6), (5→7), (6→4), (7→4), (2→8), (4→8)

39. IDENTIFICACIÓN DE LAS RUTAS (Camino basico)

RUTAS

Rutas posibles desde el grafo:

- **R1:** 1 → 2 → No array → 3
- **R2:** 1 → 2 → No o tras iterar → 8
- **R2:** 1 → 2 → Es array → 4 → 5 (rabbit no válido o no coincide) → 7 → 4 (repetición) → 8
- **R3:** 1 → 2 → Es array → 4 → 5 (rabbit válido y coincide) → 6 → 4 (repetición) → 8

40. COMPLEJIDAD CICLOMÁTICA

Se puede calcular de las siguientes formas:

- $V(G) = \text{número de nodos predicados(decisiones)} + 1$
 Número de nodos predicados(decisiones) = (isArray, filtro coincide, iterar) = 3
 $V(G) = 3 + 1 = 4$
- $V(G) = A - N + 2$
 $V(G) = 10 - 8 + 2 = 2 + 2 = 4$

DONDE:

P: Número de nodos predicado

A: Número de aristas

N: Número de nodos

Prueba caja blanca de REQ012 – Editar datos de un conejo

41. CÓDIGO FUENTE

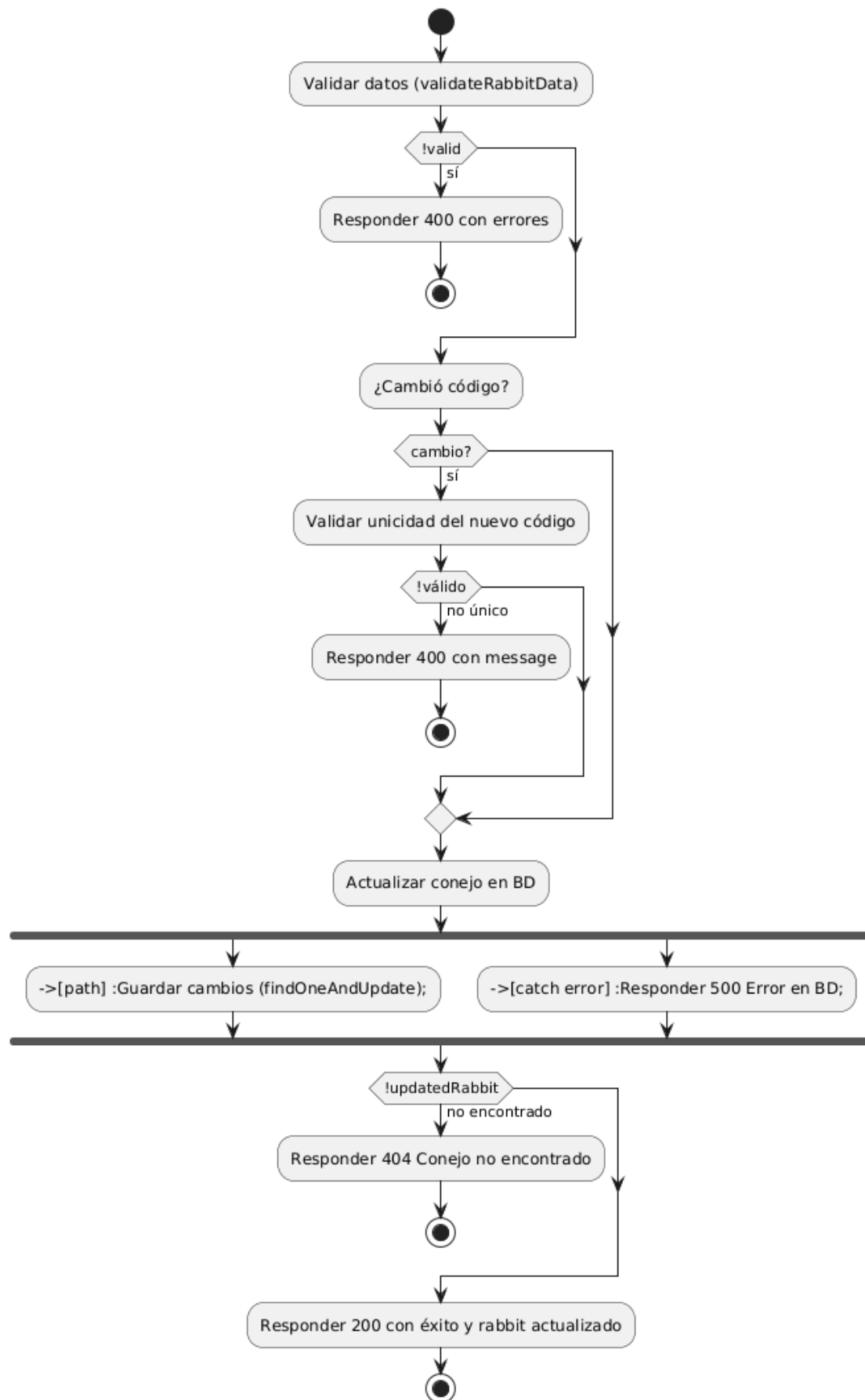
```
exports.editRabbit = async (req, res) => {
  const { code } = req.params;
  const { race, sex, age, weight, purpose } = req.body;

  try {
    // Validar datos
    const validation = await validateRabbitData(req.body);
    if (!validation.valid) {
      return res.status(400).json({ errors: validation.errors
    });
    }

    // Si el código cambió, validar que el nuevo código sea único
    if (req.body.code && req.body.code !== code) {
      const codeValidation = await
validateCodeUniqueness(req.body.code, code);
      if (!codeValidation.valid) {
        return res.status(400).json({ message:
codeValidation.message });
      }
    }

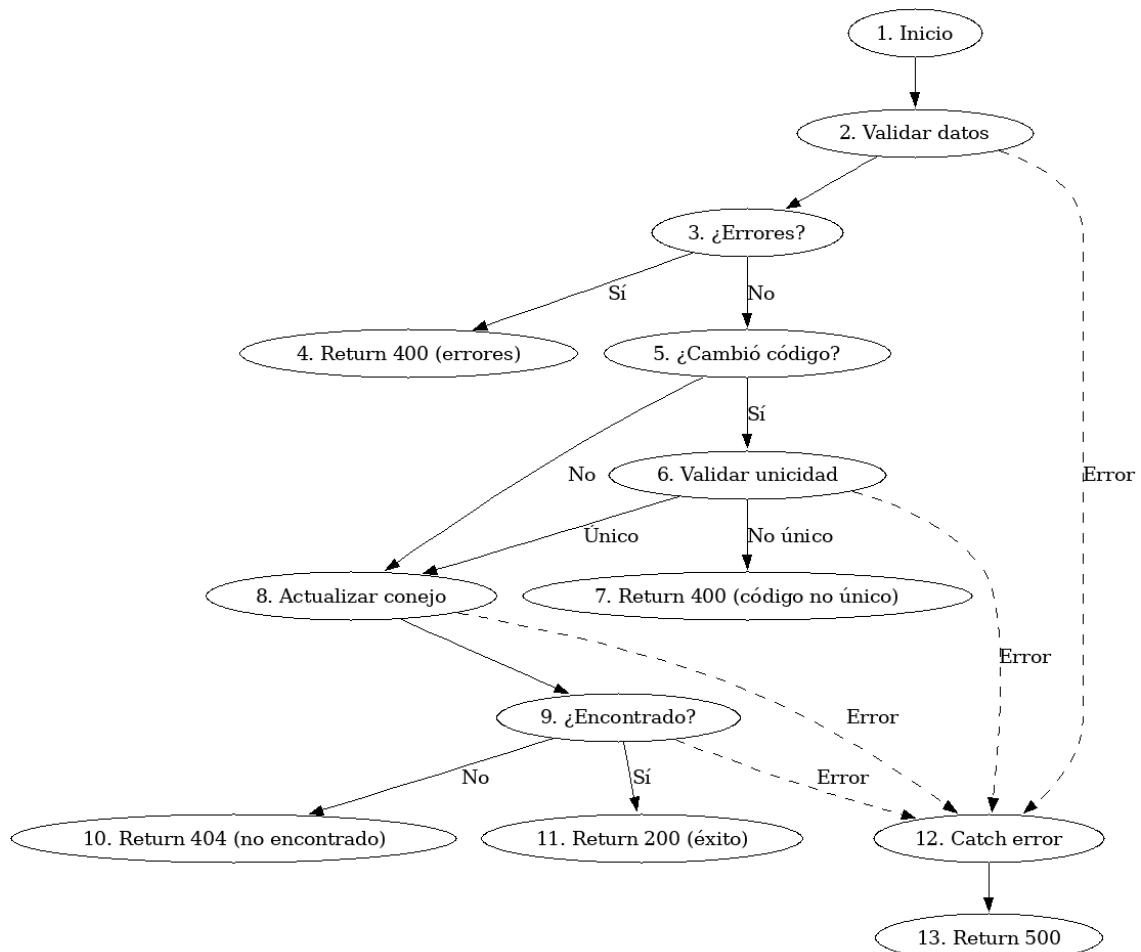
    const updatedRabbit = await Rabbit.findOneAndUpdate(
      { code },
      { race, code: req.body.code || code, sex, age, weight,
purpose },
      { new: true }
    );
    if (!updatedRabbit) {
      return res.status(404).json({ message: 'Conejo no
encontrado' });
    }
    res.status(200).json({ message: 'Conejo actualizado
exitosamente', rabbit: updatedRabbit });
  } catch (error) {
    res.status(500).json({ message: 'Error al actualizar el
conejo', error: error.message });
  }
};
```

42. DIAGRAMA DE FLUJO (DF)



43. GRAFO DE FLUJO (GF)

Nodos numerados para el grafo:



Aristas (A):

- (1→2), (2→3), (3→4), (3→5), (5→6), (5→8), (6→7), (6→8), (8→9), (9→10), (9→11), (2→12), (6→12), (8→12), (9→12), (12→13)

44. IDENTIFICACIÓN DE LAS RUTAS (Camino basico)

RUTAS

Rutas posibles desde el grafo:

- **R1:** 1 → 2 → 3 (con error) → 4
- **R2:** 1 → 2 → 3 (sin error) → 5 → 6 (cambio y código no único) → 7
- **R3:** 1 → 2 → 3 (sin error) → 5 → 6 (cambio y código único) → 8 → 9 (no encontrado) → 10
- **R4:** 1 → 2 → 3 (sin error) → 5 → 6 (no cambio) → 8 → 9 (encontrado) → 11
- **R5:** 1 → 2 → Error → 12 → 13
- **R6:** 1 → 2 → 3 → 5 → 6 → Error → 12 → 13
- **R7:** 1 → 2 → 3 → 5 → 8 → Error → 12 → 13
- **R8:** 1 → 2 → 3 → 5 → 6 → 8 → 9 → Error → 12 → 13

45. COMPLEJIDAD CICLOMÁTICA

Se puede calcular de las siguientes formas:

- $V(G) = \text{número de nodos predicados(decisiones)} + 1$
 Número de nodos predicados(decisiones) = (validación datos, cambio código, unicidad, encontrado) = 4
 $V(G) = 4 + 1 = 5$
- $V(G) = A - N + 2$
 $V(G) = 16 - 3 + 2 = 3 + 2 = 5$

DONDE:

P: Número de nodos predicado

A: Número de aristas

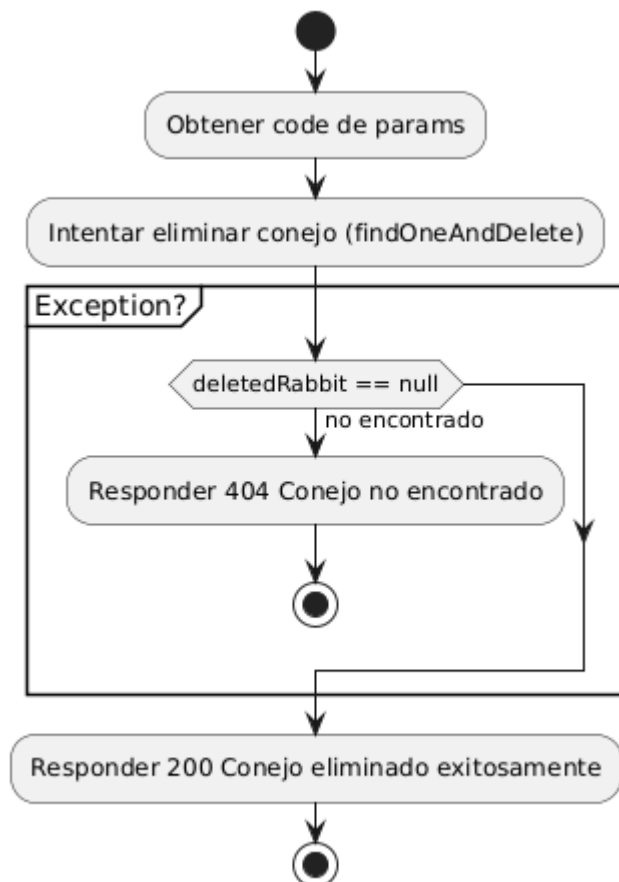
N: Número de nodos

Prueba caja blanca de REQ013 – Eliminar conejo

46. CÓDIGO FUENTE

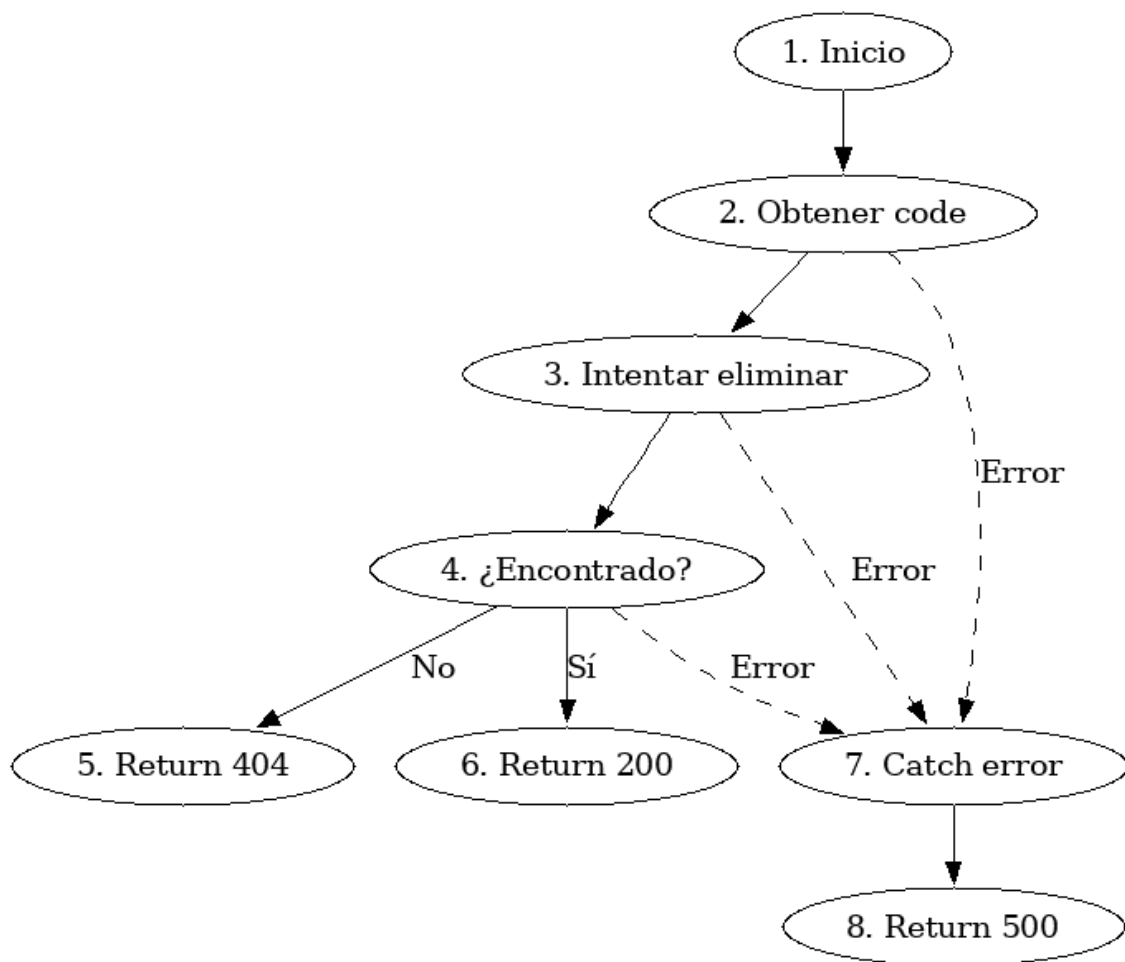
```
exports.deleteRabbit = async (req, res) => {  
  const { code } = req.params;  
  
  try {  
    const deletedRabbit = await Rabbit.findOneAndDelete({ code });  
    if (!deletedRabbit) {  
      return res.status(404).json({ message: 'Conejo no  
encontrado' });  
    }  
    res.status(200).json({ message: 'Conejo eliminado  
exitosamente' });  
  } catch (error) {  
    res.status(500).json({ message: 'Error al eliminar el conejo',  
error: error.message });  
  }  
};
```

47. DIAGRAMA DE FLUJO (DF)



48. GRAFO DE FLUJO (GF)

Nodos numerados para el grafo:



Aristas (A):

- (1→2), (2→3), (3→4), (4→5), (4→6), (2→7), (3→7), (4→7), (7→8)

49. IDENTIFICACIÓN DE LAS RUTAS (Camino básico)

RUTAS

Rutas posibles desde el grafo:

- **R1:** 1 → 2 → 3 → 4 (no encontrado) → 5
- **R2:** 1 → 2 → 3 → 4 (encontrado) → 6
- **R3:** 1 → 2 → Error → 7 → 8
- **R4:** 1 → 2 → 3 → Error → 7 → 8
- **R5:** 1 → 2 → 3 → 4 → Error → 7 → 8

50. COMPLEJIDAD CICLOMÁTICA

Se puede calcular de las siguientes formas:

- $V(G) = \text{número de nodos predicados(decisiones)} + 1$
Número de nodos predicados(decisiones) = (encontrado, error de eliminación) = 2
 $V(G) = 2 + 1 = 3$
- $V(G) = A - N + 2$
 $V(G) = 9 - 8 + 2 = 1 + 2 = 3$

DONDE:

P: Número de nodos predicado

A: Número de aristas

N: Número de nodos

Prueba caja blanca de REQ014 – Registrar datos de monta

51. CÓDIGO FUENTE

```
exports.registerMating = async (req, res) => {
  try {
    const { rabbitCode, cageNumber, matingDate } = req.body;
    if (!rabbitCode || !cageNumber || !matingDate) {
      return res.status(400).json({ message: 'Todos los campos son obligatorios.' });
    }
    // Validar que la coneja existe y cumple requisitos
    const rabbit = await Rabbit.findOne({ code: rabbitCode, sex: 'hembra', age: { $gte: 4 } });
    if (!rabbit) {
      return res.status(400).json({ message: 'La coneja seleccionada no es válida.' });
    }
    // Validar que tiene jaula asignada
    const assignment = await AssignRabbit.findOne({ rabbitCode, status: 'asignado' });
    if (!assignment || assignment.cageNumber !== cageNumber) {
      return res.status(400).json({ message: 'La coneja no tiene jaula asignada.' });
    }
    // Validar que no tenga monta activa (fecha de parto pendiente)
    const activeMating = await Mating.findOne({
      rabbitCode,
      status: 'activo',
      birthDate: { $gte: new Date() }
    });
    if (activeMating) {
      return res.status(400).json({ message: 'La coneja ya tiene una monta activa.' });
    }
    // Validar fecha de monta
    if (!matingDate) {
      return res.status(400).json({ message: 'Debe ingresar la fecha de monta.' });
    }
    const matingDateObj = new Date(matingDate);
    if (isNaN(matingDateObj.getTime())) {
      return res.status(400).json({ message: 'La fecha de monta no es válida.' });
    }
    // Calcular fecha estimada de parto (30 días después)
    const birthDate = new Date(matingDateObj);
    birthDate.setDate(birthDate.getDate() + 30);

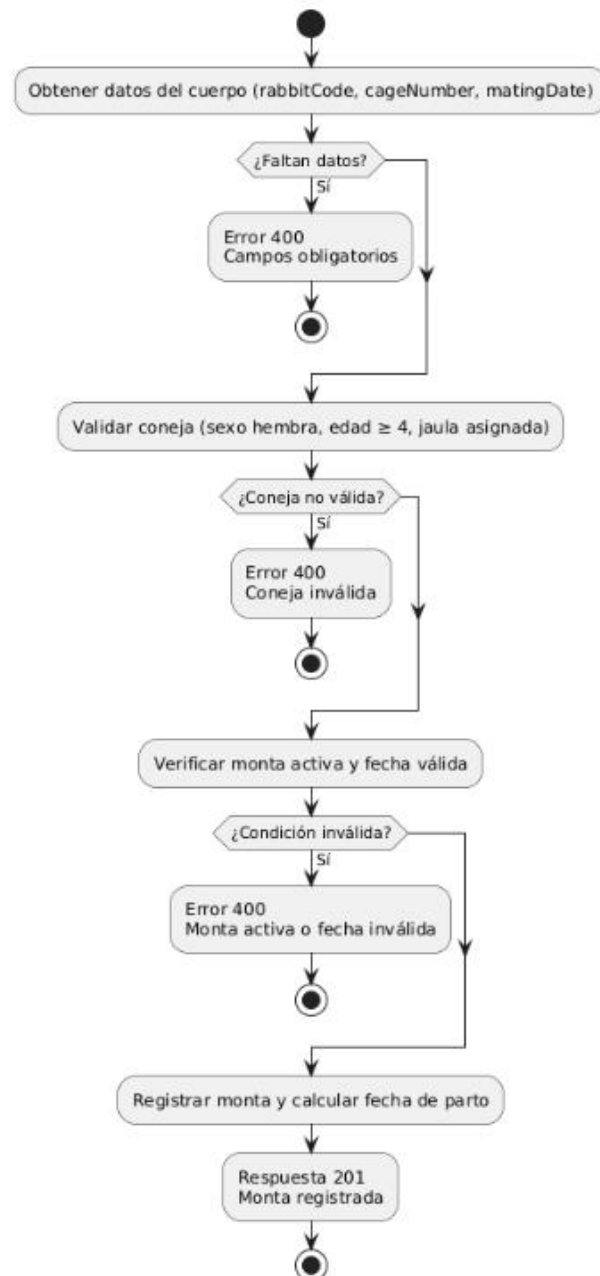
    // Registrar
```

```

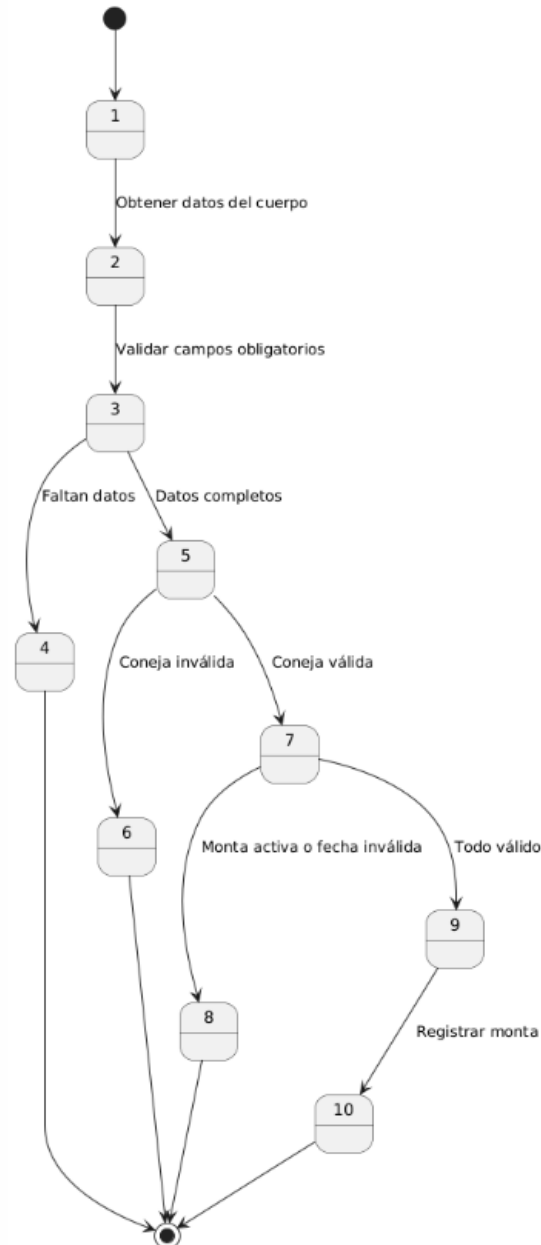
const newMating = new Mating({
  rabbitCode,
  cageNumber,
  matingDate: matingDateObj,
  birthDate,
  status: 'activo'
});
await newMating.save();
res.status(201).json({ message: 'Se registró la fecha de monta.',
mating: newMating });
} catch (err) {
  res.status(500).json({ message: 'Error al registrar la monta.',
error: err.message });
}
};

```

52. DIAGRAMA DE FLUJO (DF)



53. GRAFO DE FLUJO (GF)



54. IDENTIFICACIÓN DE LAS RUTAS (Camino basico)

Rutas posibles desde el grafo:

- **R1:** $1 \rightarrow 2 \rightarrow 3 \rightarrow 4$
Faltan datos obligatorios $\rightarrow$ error 400.
- **R2:** $1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 6$
Coneja inválida (no cumple requisitos) $\rightarrow$ error 400.
- **R3:** $1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 7 \rightarrow 8$
Fecha inválida o monta activa $\rightarrow$ error 400.
- **R4:** $1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 7 \rightarrow 9 \rightarrow 10$
Todo válido $\rightarrow$ monta registrada exitosamente.

55. COMPLEJIDAD CICLOMÁTICA

Se puede calcular de las siguientes formas:

- $V(G) = \text{número de nodos predicho (decisiones)} + 1$
- $V(G) = 3 + 1 = 4$
(Predicados: Validar campos, validar coneja, validar fecha/monta activa)

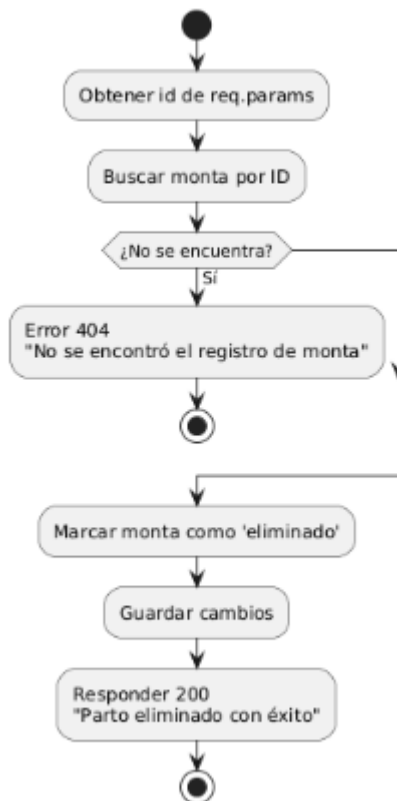
- $V(G) = A - N + 2$
- $V(G) = 9 - 7 + 2 = 4$
(A = 9 aristas, N = 7 nodos únicos)

Prueba caja blanca de REQ015 – Eliminar parto

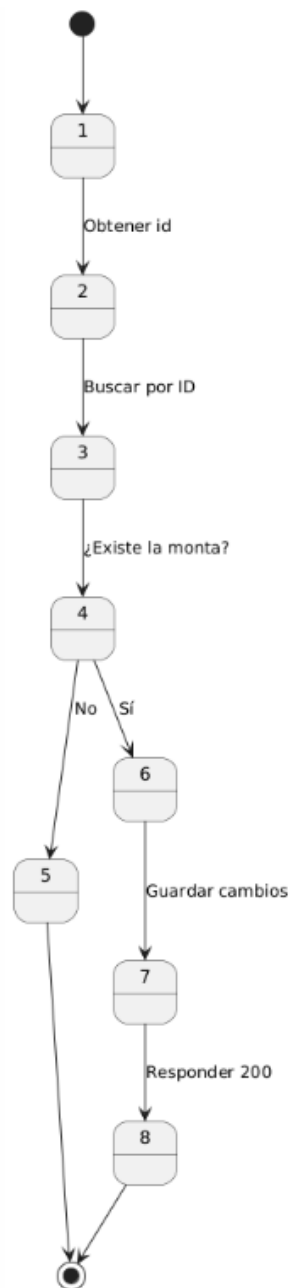
56. CÓDIGO FUENTE

```
exports.deleteMating = async (req, res) => {
  try {
    const { id } = req.params;
    const mating = await Mating.findById(id);
    if (!mating) {
      return res.status(404).json({ message: 'No se encontró el registro de monta.' });
    }
    mating.status = 'eliminado';
    await mating.save();
    res.json({ message: 'El parto ha sido eliminado con éxito.' });
  } catch (err) {
    res.status(500).json({ message: 'Error al eliminar el parto.', error: err.message });
  }
};
```

57. DIAGRAMA DE FLUJO (DF)



58. GRAFO DE FLUJO (GF)



59. IDENTIFICACIÓN DE LAS RUTAS (Camino basico)

Rutas posibles desde el grafo:

- **R1:** $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$
No se encuentra la monta $\rightarrow$ error 404.
- **R2:** $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 6 \rightarrow 7 \rightarrow 8$
Monta encontrada $\rightarrow$ se marca como eliminada $\rightarrow$ éxito 200.

60. COMPLEJIDAD CICLOMÁTICA

Se puede calcular de las siguientes formas:

- $V(G) = \text{número de nodos prediado (decisiones)} + 1$
 $V(G) = 1 + 1 = 2$
(Predicado: ¿Existe la monta?)
- $V(G) = A - N + 2$
 $V(G) = 7 - 6 + 2 = 3$

DONDE:

P: Número de nodos prediado = 1

A: Número de aristas = 7

N: Número de nodos = 6